

Verzovací systémy - Git a GitHub

Co je verzovací systém

- **verzovací systém** = nástroj pro sledování změn v souborech
- nejčastěji se používá při vývoji softwaru
- ukládá historii změn projektu
- umožňuje vrátit se ke starší verzi
- pomáhá při spolupráci více lidí na jednom projektu

Řeší například:

- kdo co změnil
- kdy to změnil
- proč to změnil
- jak se vrátit ke starší verzi
- jak pracovat na více částech projektu najednou
- jak spolupracovat v týmu

Příklad problému bez verzování:

```
projekt_final.zip  
  
projekt_final2.zip  
  
projekt_opraveno.zip  
  
projekt_opravdu_final.zip
```

Git

Co je Git

- **Git** = verzovací systém
- vytvořil ho Linus Torvalds
- původně vznikl pro vývoj Linux kernelu
- je distribuovaný
- každý vývojář má u sebe kompletní kopii repozitáře
- funguje i offline
- změny se dají později poslat na vzdálený server

Repozitář

- **repozitář** = složka projektu spravovaná Gitem
- obsahuje:
 - soubory projektu
 - historii změn
 - větve
 - informace o commitech
- Git si data ukládá do skryté složky `.git`

Složka `.git`

- skrytá složka v kořeni projektu
- obsahuje databázi Gitu
- obsahuje historii projektu
- obsahuje informace o větvích
- pokud se smaže, projekt ztratí historii verzování
- samotné soubory zůstanou, ale projekt přestane být git repozitářem

Centralizované a distribuované verzovací systémy

Centralizovaný systém

- existuje jeden centrální server
- uživatelé si ze serveru stahují aktuální verzi
- historie je hlavně na serveru
- při výpadku serveru nebo internetu je práce omezená

Příklad:

- SVN / Subversion

Distribuovaný systém

- každý vývojář má lokálně kompletní kopii repozitáře
- má i historii projektu
- může commitovat offline
- server slouží hlavně ke sdílení změn mezi lidmi

Příklad:

- Git

Jak Git ukládá změny

Snapshoty

- Git neukládá jen rozdíly mezi soubory jako jednoduchý seznam změn

- Git pracuje se **snapshoty**
- commit představuje stav projektu v daném okamžiku
- pokud se soubor nezměnil, Git si ho zbytečně neukládá znovu
- použije odkaz na už uloženou verzi

Jednoduše:

- commit = uložený bod v historii projektu
- podobá se checkpointu ve hře

Git workflow

Soubor v Gitu může být v několika stavech.

Working directory

- pracovní složka
- místo, kde reálně upravujeme soubory
- změny ještě nemusí být připravené ke commitu

Staging area

- přípravná oblast
- sem dáváme změny pomocí `git add`
- určujeme, co přesně chceme zahrnout do dalšího commitu

Repository

- uložená historie projektu
- změny se do ní uloží pomocí `git commit`

Zjednodušeně:

```
working directory → git add → staging area → git commit → repository
```

Základní příkazy Gitu

Vytvoření repozitáře

```
git init
```

- vytvoří nový git repozitář v aktuální složce
- vznikne skrytá složka `.git`

Stažení existujícího repozitáře

```
git clone <url>
```

- stáhne existující repozitář ze serveru
- vytvoří lokální kopii projektu

Příklad:

```
git clone https://github.com/uzivatel/projekt.git
```

Zjištění stavu

```
git status
```

- ukáže aktuální stav repozitáře
- například:
 - upravené soubory
 - nové nesledované soubory
 - soubory ve staging area
 - aktuální větev

Přidání změn do staging area

```
git add README.md
```

- přidá konkrétní soubor do staging area

```
git add .
```

- přidá všechny změny v aktuální složce a podsložkách

Vytvoření commitu

```
git commit -m "Popis změny"
```

- uloží připravené změny do historie
- commit by měl mít krátkou a výstižnou zprávu

Příklad:

```
git commit -m "Přidán README soubor"
```

Zobrazení historie

```
git log
```

- zobrazí historii commitů
- ukazuje například:
 - hash commitu
 - autora
 - datum
 - zprávu commitu

Kratší výpis:

```
git log --oneline
```

Porovnání změn

```
git diff
```

- ukáže rozdíly mezi verzemi
- často ukáže změněné řádky

Použití:

- před commitem
- při kontrole změn
- při hledání chyby

Návrat ke změnám

```
git restore
```

```
git restore soubor.txt
```

- zahodí neuložené změny v souboru
- vrátí soubor do stavu posledního commitu

Pozor:

- neuložené změny se tím ztratí

```
git revert
```

```
git revert <hash_commitu>
```

- vytvoří nový commit, který zruší změny staršího commitu
- je bezpečný pro sdílené repozitáře
- nemaže historii

git reset

```
git reset --hard <hash_commitu>
```

- posune historii zpět na daný commit
- může smazat novější commity z lokální historie
- používá se opatrně
- není vhodný bez rozmyslu u sdílených větví

Větve

Co je větev

- **branch** = větev vývoje
- umožňuje pracovat na změnách odděleně od hlavní verze
- hlavní větev se dnes často jmenuje `main`
- dříve se často používal název `master`

Použití větví:

- vývoj nové funkce
- oprava chyby
- experiment
- práce více lidí současně

Výpis větví

```
git branch
```

- zobrazí lokální větve
- aktuální větev bývá označena hvězdičkou

Vytvoření větve

```
git branch nova-funkce
```

- vytvoří novou větev
- ještě se do ní nepřepne

Přepnutí větve

```
git switch nova-funkce
```

- přepne se do existující větve

Starší zápis:

```
git checkout nova-funkce
```

Vytvoření a přepnutí do větve najednou

```
git switch -c nova-funkce
```

Starší zápis:

```
git checkout -b nova-funkce
```

Sloučení větve

```
git merge nova-funkce
```

- sloučí zadanou větev do aktuální větve
- nejdřív se obvykle přepneme do větve, kam chceme změny sloučit

Příklad:

```
git switch main
```

```
git merge nova-funkce
```

Merge conflict

- **merge conflict** = konflikt při slučování větví
- vznikne, když Git neumí rozhodnout, která změna je správná
- typicky když dva lidé upraví stejný řádek
- konflikt se musí opravit ručně

Postup:

- otevřít konfliktní soubor
- vybrat správnou verzi
- odstranit značky konfliktu

- přidat opravený soubor přes `git add`
- dokončit merge commitem

Vzdálené repozitáře

Co je remote

- **remote** = vzdálený repozitář
- repozitář uložený na serveru
- slouží ke sdílení změn mezi lidmi

Příklady služeb:

- GitHub
- GitLab
- Bitbucket

Přidání vzdáleného repozitáře

```
git remote add origin <url>
```

- propojí lokální repozitář se serverem
- `origin` je běžný název pro hlavní vzdálený repozitář

Odeslání změn na server

```
git push
```

- odešle lokální commity na vzdálený repozitář

První push nové větve:

```
git push -u origin main
```

Stažení změn ze serveru

```
git pull
```

- stáhne změny ze vzdáleného repozitáře
- rovnou je sloučí do aktuální větve

Pouze stažení informací

`git fetch`

- stáhne informace a změny ze serveru
- nesloučí je automaticky do aktuální větve
- umožňuje nejdřív zkontrolovat, co se změnilo

GitHub

Co je GitHub

- **GitHub** = webová služba pro hosting git repozitářů
- není to totéž co Git
- Git je nástroj pro verzování
- GitHub je služba, která repozitáře ukládá online
- umožňuje spolupráci vývojářů

Rozdíl Git vs. GitHub

Git	GitHub
verzovací systém	webová služba
běží lokálně na počítači	běží online
umí commitovat, větvit, slučovat	hostuje repozitáře
lze používat bez internetu	pro sdílení potřebuje internet

Funkce GitHubu

Fork

- **fork** = kopie cizího repozitáře na vlastní účet
- používá se hlavně u open-source projektů
- umožňuje upravit projekt bez zásahu do původního repozitáře

Pull request

- **pull request** = žádost o začlenění změn
- autor změn navrhne své úpravy
- správce repozitáře je může:
 - zkontrolovat
 - okomentovat
 - schválit
 - sloučit

Použití:

- code review
- spolupráce v týmu
- open-source vývoj

Issues

- **issues** = úkoly, chyby nebo návrhy
- používají se pro plánování práce
- mohou obsahovat:
 - popis chyby
 - návrh nové funkce
 - diskusi
 - štítky

GitHub Actions

- nástroj pro automatizaci
- často se používá pro **CI/CD**
- může automaticky spouštět:
 - testy
 - build aplikace
 - kontrolu kódu
 - nasazení aplikace

Příklad:

- po každém `push` se automaticky spustí testy

`.gitignore`

- soubor určující, co Git nemá sledovat
- používá se pro soubory, které nechceme commitovat

Typicky:

- hesla
- `.env`
- `node_modules`
- `bin`
- `obj`
- dočasné soubory
- build výstupy

Příklad `.gitignore` :

node_modules/

.env

bin/

obj/

Práce přes CLI

Co je CLI

- **CLI** = command-line interface
- práce přes příkazovou řádku
- u Gitu je velmi běžná
- umožňuje používat všechny příkazy přímo

Příklady:

- PowerShell
- Command Prompt
- Git Bash
- Terminal

Výhody:

- rychlé ovládání
- přesná kontrola
- dostupné téměř všude
- vhodné pro automatizaci

Klientské aplikace

- umožňují pracovat s Gitem graficky
- vhodné pro začátečníky
- zobrazují změny, větve a historii přehledně
- často umí řešit merge konflikty vizuálně

Příklady:

- GitHub Desktop
- GitKraken
- SourceTree
- Git klient ve VS Code
- Git integrace ve Visual Studiu

Výhody:

- přehledné rozhraní
- méně nutnosti pamatovat příkazy
- vizuální historie
- snadnější řešení konfliktů

Nevýhody:

- někdy zakrývají, co se opravdu děje
- ne všechny pokročilé operace jsou tak pohodlné
- je dobré rozumět i CLI

Praktická úloha: vytvoření repozitáře

1. Vytvoření složky a repozitáře

```
mkdir MaturitaProjekt  
  
cd MaturitaProjekt  
  
git init
```

2. Vytvoření souboru README

```
echo "# Můj Projekt" > README.md
```

3. Kontrola stavu

```
git status
```

- soubor bude pravděpodobně jako **untracked**

4. Přidání souboru do staging area

```
git add README.md
```

5. Vytvoření prvního commitu

```
git commit -m "Initial commit: přidán README"
```

6. Změna souboru

```
echo "Další text v souboru" >> README.md
```

7. Zobrazení rozdílů

```
git diff
```

- ukáže, co se změnilo oproti poslednímu commitu

8. Přidání a commit změny

```
git add .
```

```
git commit -m "Update README"
```

9. Zobrazení historie

```
git log --oneline
```

Praktická úloha: práce s větví

1. Vytvoření a přepnutí do větve

```
git switch -c nova-vetev
```

2. Úprava souboru

```
echo "Text v nové větvi" >> README.md
```

3. Commit ve větvi

```
git add .
```

```
git commit -m "Změna v nové větvi"
```

4. Návrat do hlavní větve

```
git switch main
```

5. Sloučení větve

```
git merge nova-vetev
```

Praktická úloha: propojení s GitHubem

1. Přidání remote

```
git remote add origin <url>
```

2. Odeslání na GitHub

```
git push -u origin main
```